

MIP_LINEAGE_MODEL.md

Mainframe Intelligence Platform

Enterprise Data & Execution Lineage Architecture

Version: 1.0

Status: Core Intelligence Layer

Classification: Strategic Platform Component

Purpose

Lineage explains how information and execution flow through a legacy system.

Lineage enables MIP to answer:

- Where did this data originate?
- How did it get here?
- What programs transformed it?
- What jobs consumed it?
- What systems depend on it?
- What breaks if it changes?

Without lineage:

Metadata → Static Understanding

With lineage:

Metadata
↓
Relationships
↓
Lineage
↓
Impact Analysis
↓
Modernization Intelligence

Executive Vision

A modernization program cannot succeed unless it understands:

Data Movement

How information travels.

Execution Flow

How processes execute.

Business Flow

How capabilities interact.

Transformation Flow

How modernization affects downstream systems.

Lineage Types

MIP supports four lineage categories.

1. Data Lineage
2. Execution Lineage
3. Business Lineage
4. Modernization Lineage

Data Lineage

Tracks movement of information.

Example:

CUSTOMER_FILE

↓

TS2LOAD

↓

CUSTOMER_MASTER

↓

TS2AUTH

↓

AUTHORIZATION_LOG

↓

SETTLEMENT_JOB

Questions Answered:

- Where did this field originate?
- What programs update it?
- What tables store it?
- Who consumes it?

Execution Lineage

Tracks runtime execution.

Example:

Scheduler

↓

Nightly Job

↓

Step 1

↓

Program A

↓

Program B

↓

Program C

Questions Answered:

- What starts execution?
- Which programs run next?
- Which batch chain is affected?

Business Lineage

Tracks business capability flow.

Example:

Card Activation

↓

Authorization

↓

Settlement

↓

Billing

Questions Answered:

- Which capability consumes outputs?
- Which domains interact?
- What business process is impacted?

Modernization Lineage

Tracks migration paths.

Example:

Authorization Capability

↓

Authorization Service

↓

API Layer

↓

Cloud Platform

Questions Answered:

- What replaces this component?
- What migration wave owns it?
- What future architecture depends on it?

Lineage Node Types

Application

Domain

Capability

Program

Job

Job Step

Transaction

Dataset

VSAM

DB2 Table

Column

API

Service Candidate

Migration Wave

Lineage Edge Types

Data Flow

READS
WRITES
PRODUCES
CONSUMES
UPDATES
TRANSFORMS

Execution Flow

CALLS
EXECUTES
STARTS
TRIGGERS
INVOKES

Business Flow

SUPPORTS
ENABLES
DEPENDS_ON

Modernization Flow

BECOMES

MIGRATES_TO

REPLACES

Canonical Lineage Path

Example:

Dataset

↓

Program

↓

Table

↓

Program

↓

Dataset

↓

Program

↓

Capability

This path becomes the foundation for:

- Impact Analysis
- Root Cause Analysis
- Modernization Planning

Data Lineage Model

Dataset Lineage

Example:

```
CUSTOMER_INPUT  
  
↓  
  
LOADJOB  
  
↓  
  
CUSTOMER_MASTER  
  
↓  
  
AUTHORIZATION  
  
↓  
  
AUTH_LOG
```

Store:

```
{  
  "source": "CUSTOMER_INPUT",  
  "target": "AUTH_LOG",  
  "path_length": 4,  
  "confidence": 0.95  
}
```

Table Lineage

Example:

```
CARD_MASTER  
  
↓  
  
AUTH_PROGRAM  
  
↓
```


SETTLEMENT_TABLE

↓

BILLING_TABLE

Questions:

- Which table feeds this table?
- What downstream tables depend on it?

Column Lineage

Future Enhancement

Example:

ACCOUNT_NUMBER

↓

CARD_MASTER.ACCOUNT_NUMBER

↓

AUTH_TABLE.ACCOUNT_NUMBER

↓

STATEMENT_TABLE.ACCOUNT_NUMBER

Critical for:

- Regulatory Reporting
- GDPR
- Compliance
- Audit

Execution Lineage Model

Batch Chain

Example:

JOB001

↓

TS2LOAD

↓

JOB002

↓

TS2AUTH

↓

JOB003

↓

TS2SETTLE

Questions:

- What batch chain depends on JOB001?
- Which downstream jobs are affected?

Online Transaction Flow

Example:

Transaction

↓

Program

↓

Program

↓

DB2

↓

Response

Questions:

- What path executes during authorization?
- Which systems participate?

Business Lineage Model

Maps technical assets to business value.

Example:

Program

↓

Capability

↓

Domain

↓

Application

Questions:

- Which capability owns this program?
- Which domain is impacted?

Lineage Construction Pipeline

Repository

↓

Metadata

↓

Relationships

↓

Knowledge Graph

↓

Lineage Builder

↓

Lineage Repository

Lineage Storage

SQLite Tables

lineage_node

lineage_edge

lineage_path

lineage_snapshot

Lineage Path Schema

```
CREATE TABLE lineage_path (  
    path_id TEXT PRIMARY KEY,  
    source_node TEXT,  
    target_node TEXT,  
    path_type TEXT,  
    hop_count INTEGER,  
    confidence_score REAL,  
    created_at TIMESTAMP  
);
```

NetworkX Lineage Construction

Use:

```
nx.MultiDiGraph()
```

Path Discovery:

```
nx.shortest_path()  
  
nx.all_simple_paths()  
  
nx.descendants()  
  
nx.ancestors()
```

Core Lineage Algorithms

Upstream Lineage

Question:

```
Where did this originate?
```

Traversal:

```
Reverse Graph Traversal
```

Downstream Lineage

Question:

```
Who depends on this?
```

Traversal:

Forward Graph Traversal

End-to-End Lineage

Question:

How does this flow through the system?

Traversal:

Source → Target Path Analysis

Lineage Confidence Model

Every lineage path receives confidence.

Factors:

Parser Confidence

Metadata Quality

Relationship Confidence

Evidence Completeness

Explainability

Every lineage result includes:

```
{
  "path": [],
  "confidence": 0.94,
  "evidence": [],
  "reasoning": []
}
```

Example Questions

Data Origin

Where did ACCOUNT_NUMBER originate?

Downstream Impact

Who consumes CUSTOMER_MASTER?

Batch Dependency

Which jobs depend on JOB001?

Capability Impact

Which business capability depends on TS2AUTH?

Enterprise Scale Targets

Support:

100,000+ Programs

20,000+ Jobs

500,000+ Tables

10M+ Relationships

Millions of Lineage Paths

Success Criteria

A new engineer should be able to ask:

Show the complete lifecycle of a card authorization transaction.

And receive:

- Programs
- Jobs
- Tables
- Datasets
- Dependencies
- Capabilities
- Modernization Impact

without reading COBOL.

MIP Principle

Metadata captures facts.

Knowledge Graph captures relationships.

Lineage captures movement.

Impact Analysis captures consequences.

Modernization Intelligence captures opportunity.

End of Document.